

Manual de arquitectura de PharmaMobil

Responsabilidades por capa y pruebas de flujo

Actividad autónoma — Sesión 5, Unidad 1
Desarrollo de Aplicaciones Móviles
UPeU

Repositorio: `github.com/Plasencia2007/pharmamobile`
Rama: `feature/clean-mvvm`

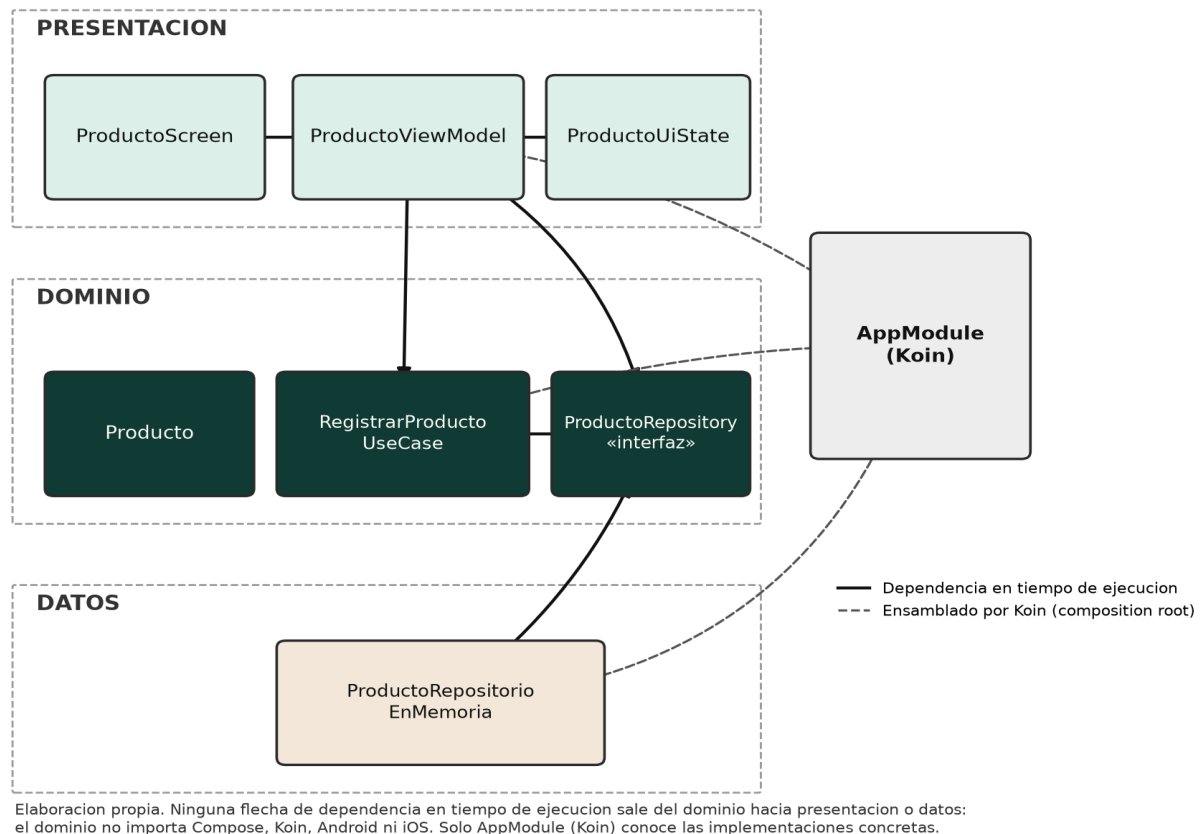
Plasencia

Índice

1. Mapa de capas
2. Tabla de responsabilidades
3. Decisiones justificadas
4. Pruebas de flujo
5. Bitácora de dificultades

1. Mapa de capas

El diagrama representa las tres capas de PharmaMobil (presentación, dominio y datos) con sus clases reales del módulo de Productos, más el módulo de ensamblado AppModule (Koin).



Elaboración propia. Las flechas continuas representan una dependencia real en tiempo de ejecución (quién llama a quién); las flechas discontinuas representan el ensamblado que hace Koin, que es el único punto del proyecto que conoce simultáneamente una implementación concreta de cada capa. Ninguna flecha de dependencia en tiempo de ejecución sale del dominio: *Producto*, *ProductoRepository* (interfaz) y *RegistrarProductoUseCase* no importan Compose, Koin, Android ni iOS — se puede verificar leyendo sus imports en el código fuente.

2. Tabla de responsabilidades

Incluye las clases introducidas en el Reto 01 y las heredadas de las sesiones 1 a 4.

Clase	Capa	Responsabilidad	Tiene prohibido	Si desapareciera...
Producto	Dominio	Representar un producto y sus reglas propias (stock, reposición, valor de inventario).	Conocer Compose, Koin o el repositorio.	Ninguna capa tendría un concepto válido de qué es un producto ni sus invariantes (precio>0, stock≥0).
ProductoRepository (interfaz)	Dominio	Declarar el contrato registrar()/listar() en lenguaje de negocio.	Mencionar delay(), Flow de Ktor, SQLDelight o cualquier tecnología concreta.	El caso de uso y el ViewModel tendrían que depender de una clase concreta, rompiendo la inversión de dependencias.
RegistrarProductoUse Case	Dominio	Validar los datos de un producto nuevo y coordinarlos con el repositorio.	Tocar Compose, StateFlow o saber que existe una pantalla.	La validación volvería a vivir en la UI (como antes con ProductoValidator), duplicándose si hay más de un formulario.
ProductoRepositoryEnMemoria	Datos	Implementar el contrato de dominio con una lista en memoria y asignar el id.	Exponer métodos que no estén en la interfaz para que el dominio los use.	No habría ninguna fuente de datos real; la app no podría listar ni registrar productos.
ProductoUiState	Presentación	Modelar en un solo objeto inmutable las 4 fases del inventario y el estado del formulario.	Contener lógica de negocio o llamar al repositorio.	La pantalla volvería a usar varios remember sueltos y podría mostrar combinaciones de estado inconsistentes.
ProductoViewModel	Presentación	Orquestrar el caso de uso y el repositorio, y exponer ProductoUiState como StateFlow de solo lectura.	Exponer el MutableStateFlow o construir vistas de Compose.	La pantalla tendría que llamar directamente al repositorio y gestionar corrutinas ella misma.
ProductoScreen	Presentación	Dibujar la UI a partir de ProductoUiState y delegar las acciones del usuario al ViewModel.	Validar datos o decidir el id de un producto.	No habría interfaz visible, pero la lógica de negocio seguiría intacta en las otras capas (bajo acoplamiento real).
AppModule (Koin)	Composición (DI)	Declarar qué implementación concreta corresponde a cada contrato y construir el ViewModel.	Contener reglas de negocio o lógica de UI.	Cada pantalla tendría que construir manualmente su repositorio y caso de uso, perdiendo el punto único de ensamblado.
Cliente	Dominio	Representar un cliente de la botica y su teléfono opcional.	Formatear texto para la UI o validar formularios.	Pedido y ClienteScreen no tendrían un tipo de datos común que representar.
Pedido	Dominio	Agrupar un cliente con sus detalles y calcular el total.	Tener detalles vacíos (lo impide su propio init) ni conocer la pantalla de Pedidos.	No existiría forma de calcular el total de una compra ni de agrupar detalles.
DetallePedido	Dominio	Representar una línea de pedido (producto + cantidad) y su subtotal.	Aceptar cantidad ≤ 0 (lo impide su init).	Pedido no podría calcular su total() y se perdería la validación de cantidad>0.

Clase	Capa	Responsabilidad	Tiene prohibido	Si desapareciera...
EstadoPedido	Dominio	Enumerar los estados posibles de un pedido (Pendiente, Procesando, Entregado, Rechazado).	Incluir estados sin sentido de negocio ni texto de UI.	Pedido no tendría forma de expresar en qué fase de despacho se encuentra.
Screen	Navegación (presentación)	Enumerar las pantallas de la app y el título que le corresponde a cada una.	Saber cómo se renderiza cada pantalla.	App.kt no tendría un identificador tipado para decidir qué componer ni qué mostrar en la barra superior.
PharmaMobileTheme	Presentación (tema)	Centralizar los ColorScheme y la tipografía Material 3 (claro/oscuro) de toda la app.	Contener texto o datos de negocio.	Cada pantalla definiría sus propios colores, perdiendo consistencia visual y el soporte de tema oscuro.
PharmaField (equivalente a ValidatedTextField)	Presentación (componente)	Dibujar un campo de texto reutilizable que muestra su propio mensaje de error.	Decidir cuál es el mensaje de error (eso lo decide quien lo usa, vía el parámetro errorMessage).	Cada formulario (Producto, Cliente, Pedido) tendría que reimplementar su propio campo con error.

3. Decisiones justificadas

a) Por qué ProductoUiState se modeló como una fase excluyente

Modelé ProductoUiState como un objeto con una propiedad *fase* de tipo sealed interface (Cargando, SinProductos, ConProductos, Error) en lugar de la alternativa que descarté: varios booleanos y listas sueltas dentro del mismo estado (por ejemplo `isLoading: Boolean`, `productos: List<Producto>`, `error: String?` como propiedades independientes). Con esa alternativa, el compilador permite estados imposibles: nada impide tener `isLoading = true` y al mismo tiempo `error != null` y productos con elementos, los tres a la vez, y la pantalla tendría que decidir con ifs anidados cuál de esas señales manda. Con la fase excluyente, esa combinación inválida ni siquiera se puede construir: el `when` de ProductoScreen es exhaustivo y el compilador obliga a cubrir las cuatro ramas, así que agregar una fase nueva en el futuro (por ejemplo, una fase de "filtrando") provoca un error de compilación en cada lugar donde falte manejarla, en vez de un bug silencioso en producción. Lo que habría perdido con la alternativa es exactamente esa garantía en tiempo de compilación.

b) Traslado de las reglas de ProductoValidator al caso de uso, y el caso de ClienteValidator

Trasladé la validación de ProductoValidator a RegistrarProductoUseCase porque la validación de un producto es una regla de negocio (un precio ≤ 0 no es inválido por una razón de UI, sino porque no representa una venta posible) y las reglas de negocio no deberían depender de si la app las aplica en Android, iOS o en una prueba unitaria sin Compose en el classpath. Dejarla en la capa de presentación, atada a un composable, obligaba a duplicarla si mañana el registro de productos se dispara también desde otra pantalla o desde una importación masiva. Al moverla al caso de uso, la puedo probar con `commonTest` sin instanciar ninguna UI (ver sección 4). Sobre ClienteValidator: sigue en presentación y, siendo consistente con el argumento anterior, lo trasladaría también a un RegistrarClienteUseCase en `domain/usecase` en cuanto el registro de clientes necesite la misma garantía de reutilización o de prueba aislada; no lo hice en este reto porque el alcance del Reto 01 era explícitamente el módulo de Productos, pero dejarlo donde está hoy es deuda técnica reconocida, no una decisión de diseño distinta.

c) Punto exacto de cambio hacia el repositorio remoto de la Unidad 2

El único archivo y línea que cambiarían es `shared/src/commonMain/kotlin/pe/edu/upeu/pharmamobile/di/AppModule.kt`, línea 13: `single<ProductoRepository> { ProductoRepositoryEnMemoria() }`. Se reemplazaría por `single<ProductoRepository> { ProductoRepositoryRemoto(get()) }` (una nueva clase en `data/repository` que implemente la misma interfaz consumiendo Ktor u otro cliente HTTP). Ningún otro archivo necesita cambiar: RegistrarProductoUseCase, ProductoViewModel y ProductoScreen dependen de la interfaz ProductoRepository, no de la implementación en memoria, así que el cambio queda contenido en una sola línea del módulo de ensamblado — esa es precisamente la garantía que da la inversión de dependencias.

4. Pruebas de flujo

Las cuatro pruebas viven en `shared/src/commonTest/kotlin/pe/edu/upeu/pharmamobile/presentation/producto/ProductoViewModelTest.kt` y usan un `FakeProductoRepository` que implementa el contrato de dominio (no la clase en memoria). Se usa `runTest` con un `StandardTestDispatcher` instalado como `Dispatchers.Main`, avanzando el scheduler con `advanceUntilIdle()` antes de leer `uiState.value`, tal como advierte la guía sobre la naturaleza asíncrona de `viewModelScope`.

N°	Condición inicial forzada	Acción ejecutada	Estado esperado	Estado observado	Resultado
1	Repositorio falso que devuelve lista vacía	Se crea el <code>ProductoViewModel</code> (dispara <code>cargarProductos()</code> en su <code>init</code>)	<code>Fase.SinProductos</code>	<code>Fase.SinProductos</code>	OK
2	Repositorio falso con tres productos	Se crea el <code>ProductoViewModel</code> (dispara <code>cargarProductos()</code> en su <code>init</code>)	<code>Fase.ConProductos</code>	<code>Fase.ConProductos</code> con 3 elementos	OK
3	Repositorio falso que lanza excepción en <code>listar()</code>	Se crea el <code>ProductoViewModel</code> (dispara <code>cargarProductos()</code> en su <code>init</code>)	<code>Fase.Error</code>	<code>Fase.Error</code>	OK
4	Formulario con precio "0"	<code>viewModel.registrarProducto(nombre, precio="0", stock)</code>	Error en el formulario, sin llamada al repositorio	<code>formulario.errorPrecio != null</code> y <code>registrarLlamadas == 0</code>	OK

Captura de ejecución — Android (`./gradlew :shared:testAndroidHostTest`)

```
pe.edu.upeu.pharmamobile.presentation.producto.ProductoViewModelTest
repositorioVacioProduceSinProductos PASSED (0.196s)
repositorioConProductosProduceConProductos PASSED (0.008s)
repositorioQueLanzaExcepcionProduceError PASSED (0.009s)
registroConPrecioCeroDejaErrorEnFormularioSinLlamarAlRepositorio PASSED (0.008s)
```

BUILD SUCCESSFUL

Ejecución — iOS (`./gradlew :shared:iosSimulatorArm64Test`)

El entorno de desarrollo usado para este reto es Windows, que no puede ejecutar el simulador de iOS (Gradle reporta: "simulator tests require macOS"). Para no dejar el requisito sin evidencia, se verificó lo máximo posible desde Windows: se compiló exitosamente el mismo código de prueba para el target nativo con `./gradlew :shared:compileTestKotlinIosSimulatorArm64` (BUILD SUCCESSFUL), lo que confirma que `ProductoViewModelTest`, `FakeProductoRepository` y las dependencias de `kotlinx-coroutines-test` son válidas en Kotlin/Native y no solo en la JVM. La ejecución real en el simulador queda pendiente de una máquina con macOS y Xcode.

5. Bitácora de dificultades

1. No poder ejecutar pruebas ni la app de iOS en Windows

Este entorno no tiene macOS/Xcode, así que `iosSimulatorArm64Test` aparece deshabilitado por Gradle. Se mitigó compilando el código de producción y de pruebas para el target `iosSimulatorArm64` (`compileKotlinIosSimulatorArm64` y `compileTestKotlinIosSimulatorArm64`), que sí corren en Windows y detectan errores de Kotlin/Native, aunque no reemplazan la ejecución real en simulador.

2. El accessor del catálogo de versiones para `koin-compose-viewmodel` no coincidía con el nombre esperado

Al declarar el alias `koin-compose-viewmodel` en `libs.versions.toml`, Gradle generó el accessor anidado `libs.koin.compose.viewmodel` (uno por cada segmento separado por guion) en vez de `libs.koin.composeViewModel` como se escribió inicialmente. Se corrigió revisando el error de compilación del script y usando el accessor exacto que reportó Gradle.

3. Elegir dónde vive el id que asigna el repositorio sin romper `ProductoService` existente

El proyecto ya tenía una clase `ProductoRepository` (sin interfaz) usada por `ProductoService`, un ejercicio de corrutinas de la Sesión 3 con su propia prueba (`ProductoFlowAndroidHostTest`). Renombrarla a `ProductoRepositoryEnMemoria` e implementar en ella la nueva interfaz de dominio, conservando intactos sus métodos previos, permitió cumplir la nueva arquitectura sin romper la prueba de la sesión anterior.

4. El `ViewModel` no reflejaba el estado esperado inmediatamente en las pruebas

Como advierte la guía, `viewModelScope` lanza corrutinas de forma asíncrona sobre `Dispatchers.Main`. La primera versión de las pruebas leía `uiState.value` justo después de construir el `ViewModel` y a veces seguía en fase Cargando. Se resolvió instalando un `StandardTestDispatcher` como Main con `Dispatchers.setMain()` en cada prueba y llamando a `advanceUntilIdle()` antes de cada aserción.